unie*
Universidad

Grado en Ingeniería Informática

Asignatura: Ingeniería del Software de Componentes y Sistemas Distribuidos
Curso académico: 2025/26
Unidad: Casos Prácticos
Actividad: 2

Planeta Formación y Universidades

Asignatura: Ingeniería del Software de Componentes y Sistemas Distribuidos (Curso 2025/26)

Profesor: Fernando Rodríguez Sela

Título de la actividad: Migración a Arquitectura de Microservicios

---

1 Descripción General

En esta práctica transformaréis el sistema monolítico desarrollado en la Práctica 1 en una **arquitectura de microservicios** desacoplados que se comunican mediante **colas de mensajes**. Esta migración os permitirá experimentar los beneficios y desafíos de distribuir la funcionalidad en servicios independientes.

1.1 ¿Por qué esta migración?

El paso de monolito a microservicios ilustra:

- **Desacoplamiento**: Servicios independientes con responsabilidades únicas
- **Escalabilidad**: Capacidad de escalar servicios individualmente
- **Resiliencia**: Tolerancia a fallos de componentes individuales
- **Mantenibilidad**: Equipos pueden trabajar en servicios separados
- **Comunicación asíncrona**: Procesamiento no bloqueante mediante mensajería

2 Objetivos de Aprendizaje

- Diseñar y descomponer un sistema monolítico en microservicios
- Implementar comunicación asíncrona mediante colas de mensajes
- Gestionar la orquestación y coordinación entre servicios distribuidos
- Implementar patrones de resiliencia (retry, circuit breaker, dead letter queues)
- Comprender los *trade-offs* entre arquitecturas monolíticas y distribuidas
- Gestionar la persistencia distribuida y consistencia eventual
- Implementar observabilidad en sistemas distribuidos (logging, tracing)

3 Arquitectura Objetivo

3.1 Descomposición en Microservicios

Debéis descomponer vuestro monolito en **al menos 5 microservicios independientes**:

3.1.1 1. API Gateway (Obligatorio)

- Punto de entrada único al sistema
- Enrutamiento de peticiones a servicios correspondientes
- Autenticación y autorización básica
- Rate limiting (opcional)
- Publica mensajes en colas y/o espera respuestas

3.1.2 2. Servicio de Base de Datos (Obligatorio)

- Gestión centralizada de persistencia
- Responde a mensajes de lectura/escritura
- Operaciones CRUD sobre entidades del sistema
- Puede mantener caché local (opcional)

3.1.3 3. Servicio LLM (Obligatorio)

- Interfaz con Ollama u otro LLM
- Procesa peticiones de generación de contenido
- Gestiona el contexto de conversación
- Publica respuestas en cola de resultados

3.1.4 4-N. Servicios de Conectores Externos (Mínimo 2)

Cada servicio externo de la Práctica 1 debe convertirse en un microservicio:

- **Servicio Telegram**: Gestión del bot y comunicación
- **Servicio Meteorológico**: Consultas a APIs del tiempo
- **Servicio de Mapas**: Geocoding y geolocalización
- **Servicio de [API elegida]**: Integración con API pública seleccionada
- Otros según los servicios implementados en Práctica 1

3.2 Sistema de Mensajería

Debéis implementar comunicación mediante colas. Se recomienda el uso de **RabbitMQ**:

- Sistema de mensajería AMQP completo
- Soporte para múltiples patrones (pub/sub, request/reply, work queues)
- Dead Letter Exchanges para gestión de errores
- Gestión de prioridades y TTL

3.3 Patrones de Comunicación Requeridos

Debéis implementar al menos 3 de estos patrones:

1. **Request/Reply**: Un RPC montado sobre AMQP
2. **Publish/Subscribe**: Se lanza un evento a la cola a la que están subscritos varios servicios
3. **Work Queue**: Conectar múltiples *workers* a la misma cola.
4. **Event Sourcing**: Notificación por eventos de cambios de estado
5. **Saga Pattern**: Cada microservicio guardará las transacciones ejecutadas y podrá deshacerlas en caso de fallo del proceso global (opcional, +bonus)

4 Requisitos Funcionales

4.1 Funcionalidad Mínima

El sistema debe **mantener toda la funcionalidad** de la Práctica 1:

- Misma temática y características funcionales
- Todas las integraciones externas operativas
- API REST pública funcional (a través del API Gateway)
- Bot de Telegram operativo (si lo implementasteis)

4.2 Nuevas Funcionalidades (Obligatorias)

4.2.1 1. Procesamiento Asíncrono

- Las peticiones al LLM se procesan de forma asíncrona
- Sistema de notificación cuando se completa el procesamiento
- Posibilidad de consultar el estado de una petición en curso

4.2.2 2. Gestión de Errores Distribuida

- Dead Letter Queue para mensajes fallidos
- Reintentos automáticos configurables
- Logging centralizado de errores

4.2.3 3. Monitorización Básica

- Endpoint de health check en cada microservicio
- Dashboard simple mostrando estado de servicios
- Métricas básicas: mensajes procesados, errores, latencia

5 Requisitos Técnicos

5.1 Arquitectura y Despliegue

- **Containerización**: Cada microservicio en su propio contenedor Docker
- **Docker Swarm o Kubernetes**: Orquestación de todos los servicios (OBLIGATORIO)
- **Redes Docker**: Separación de redes (frontend, backend, data)
- **Variables de entorno**: Configuración externalizada
- **Volúmenes**: Persistencia de datos de BD y colas

5.2 Sugenrencia Tecnologías

- **Lenguaje base**: Node.js (puede combinarse con Python para servicios específicos)
- **Framework web**: Fastify o Express
- **Mensajería**: RabbitMQ (amqplib)
- **Base de datos**: La misma de Práctica 1
- **Logging**: Winston, Pino o similar
- **Docker**: Docker + Docker Compose

6 Entregables

1. **Código fuente completo** en repositorio Git y en el campus virtual
2. **Archivos Dockerfile** que generen cada contenedor
3. **Documentación completa**:
   - `README.md`: Instrucciones de instalación y ejecución
   - `docs/arquitectura`: Diagrama y explicación de la arquitectura
   - `docs/mensajes`: Esquema de mensajes y colas utilizadas
   - `docs/api`: Documentación de la API del Gateway
   - `docs/migracion`: Decisiones de diseño y proceso de migración
4. **Vídeo demo** (10-15 minutos) mostrando:
   - Despliegue completo
   - Funcionamiento del sistema end-to-end
   - Visualización de mensajes en las colas
   - Comportamiento ante fallos (matar un servicio)
   - Monitorización y logs

5. **Informe técnico** (PDF, 5-8 páginas):
   - Comparativa monolito vs microservicios en vuestro caso
   - Decisiones de descomposición y justificación
   - Patrones de mensajería implementados
   - Estrategias de gestión de errores y resiliencia
   - Desafíos encontrados y soluciones aplicadas
   - Conclusiones y lecciones aprendidas

7 Criterios de Evaluación

Criterio	Peso
Descomposición correcta en microservicios (5+)	20 %
Implementación de mensajería asíncrona	20 %
Contenerización y orquestación funcional y bien configurado	15 %
Gestión de errores y resiliencia	10 %
Mantenimiento de funcionalidad de Práctica 1	10 %
Documentación de arquitectura y diseño	10 %
Calidad del código y organización	10 %
Informe técnico y análisis comparativo	5 %

8 Guía de Migración

8.1 Fase 1: Planificación

1. Analizar el monolito y identificar responsabilidades
2. Diseñar la arquitectura de microservicios
3. Definir esquema de mensajes y colas
4. Diseñar el modelo de datos distribuido

8.2 Fase 2: Infraestructura

1. Configurar RabbitMQ
2. Crear estructura de proyecto multi-servicio
3. Configurar Docker Swarm o Kubernetes
4. Implementar librerías compartidas (mensaje, logging)

8.3 Fase 3: Implementación de Servicios

1. Desarrollar API Gateway
2. Migrar cada conector a su microservicio
3. Implementar servicio de base de datos
4. Implementar servicio LLM
5. Conectar servicios mediante mensajería

8.4 Fase 4: Resiliencia y Testing

1. Implementar gestión de errores y reintentos
2. Configurar Dead Letter Queues
3. Añadir health checks y monitorización
4. Testing de integración completo
5. Pruebas de resiliencia (chaos testing)

8.5 Fase 5: Documentación y Entrega

1. Documentar arquitectura y decisiones
2. Preparar vídeo demo
3. Escribir informe técnico
4. Revisar código y documentación

9 Consideraciones Importantes

9.1 Diseño de Mensajes

Definid un esquema claro de mensajes. Ejemplo:

[language=json] "messageId": "uuid-v4", "timestamp": "ISO-8601", "type": "LLM$_{R}EQUEST$", "$correla$
"$request-uuid$", "$payload$" : "$prompt$" : "...", "$userId$" : "...", "$context$" : ..., "$metadata$" :
"$source$" : "$api-gateway$", "$priority$" : "$normal$"

9.2 Gestión de Estado

- Los microservicios deben ser **stateless** en lo posible
- El estado se guarda en la base de datos o caché
- Usar `correlationId` para rastrear peticiones
- Considerar idempotencia en operaciones

9.3 Resiliencia

Implementad al menos:

- **Reintentos exponenciales**: Con backoff para mensajes fallidos
- **Dead Letter Queue**: Para mensajes que fallan repetidamente
- **Timeouts**: En todas las operaciones de mensajería
- **Health checks**: Para monitorizar estado de servicios

9.4 Problemas Comunes y Soluciones

Problema	Solución
Peticiones lentas por múltiples saltos	Implementar request/reply con timeout
Pérdida de mensajes	Usar acknowledgments y persistencia
Orden de mensajes no garantizado	Usar colas FIFO o añadir secuencia
Duplicados de mensajes	Implementar idempotencia
Servicios caídos	Health checks + auto-restart
Debugging difícil	Logging correlacionado + tracing

10 Criterios de Éxito

Vuestro sistema estará listo para entregar cuando:

- Se levanta completamente
- Todos los servicios se comunican correctamente por mensajería
- La funcionalidad de Práctica 1 está operativa
- Sobrevive al fallo de un servicio individual (exceptuando Gateway y RabbitMQ)
- Los logs muestran el flujo de mensajes claramente
- La documentación permite a otra persona entender y desplegar el sistema

11 Preguntas Frecuentes

¿Puedo usar otros lenguajes además de Node.js?
Sí, podéis implementar servicios específicos en Python, Go, etc.

¿Necesito reimplementar todo desde cero?
No, podéis reutilizar mucho código de Práctica 1, reorganizándolo en servicios.

¿Qué pasa si RabbitMQ o Redis se cae?
El sistema debe degradarse gracefully. Los servicios deben reintentar la conexión.

¿Puedo trabajar en parejas?
Si, mantenéis los mismos equipos que en la primera práctica